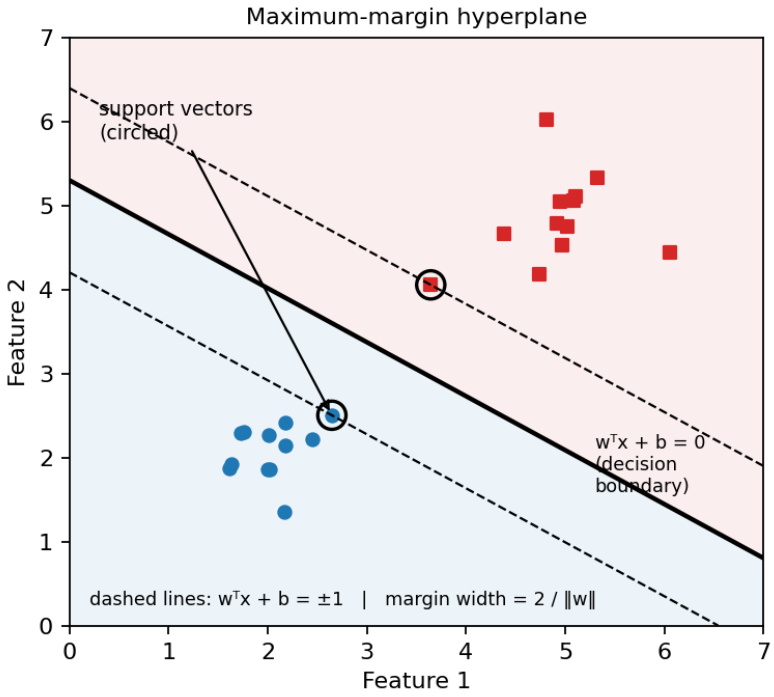
STA 3402 – Machine Learning & Neural Networks

Support Vector Machines (SVM) and Support Vector Regression (SVR)

Classification, regression and kernels + Introduction to Neural Networks

Lecture notes – Day 7 (13th) • simple English • with R code, real R outputs, plots and worked exercises



What is inside

Part	Topic
0	Class notes (what was done, homework, lab, group project)
1	Recap: supervised learning, train/test split, cross-validation, other ways to "draw lines"
2	Hyperplanes and linear classifiers
3	Which line is best? The maximum-margin idea and support vectors
4	Hard-margin SVM (the maths, simply)
5	Noisy data, soft margin, slack variables, the cost C, hinge loss
6	The dual problem (why support vectors matter)
7	Multi-class SVM: the one-vs-one trick
8	Non-linear SVM and the kernel trick (linear, polynomial, RBF, sigmoid)
9	Support Vector Regression (ε-SVR) and the ε-tube
10	Data preparation, tuning without leakage, evaluation metrics
11	R demo: SVM classification (linear, RBF, tuning, probabilities, class weights)
12	R demo: Support Vector Regression
13	In-class exercises (iris versicolor vs virginica; mtcars lm vs SVR) + homework
14	Common mistakes and comparison with other methods
15	Exercises 1–7 with worked solutions
16	Introduction to Neural Networks
17	Key takeaways and R cheat-sheet

0. Class notes

- Last class (on the 6th): **recommendation systems** + exercises and a classroom exercise. All files are uploaded to the LMS.
- Today: a new algorithm – **Support Vector Machine (SVM)** for classification and **Support Vector Regression (SVR)** for regression. Then a start on **Neural Networks**.
- According to the syllabus, SVM and kernels were not covered in the earlier course, so they are included here. After SVM, the machine-learning part is finished; what is left is **neural networks and deep learning**.
- **Group project:** the lecturer plans to ask you to use **Python**, so you get coverage of both R and Python. Instructions will come on the LMS.
- **Assignment (part 2):** remember the first Kaggle assignment where you used basic regression models? After learning advanced models (like SVR), you will revisit that problem to improve your results.
- **Homework:** for the iris exercise, split the data first, then compare SVM with **logistic regression** and **random forest**. An upload link will be on the LMS.
- **Lab sheet** will be posted on the LMS – do it on **Monday**. Attend lab sessions regularly.
- Lecturer's message: "You should be sitting in front of a computer from day one. If you don't code, there is no learning. After marks are released, don't email me begging – you have to earn your grade by practising."

Interview tip

Questions like "*SVM works for two classes – how do you use it for many classes?*" are common **technical interview** questions. Recently, data-science students going for internships had two technical interviews and were asked "inside-out". You must know **what is happening**, not only how to run the code.

1. Recap: supervised learning and "drawing lines"

1.1 Supervised learning

SVM and SVR are **supervised learning** methods: the **label** (target / response variable) is available, so the model can "learn under supervision".

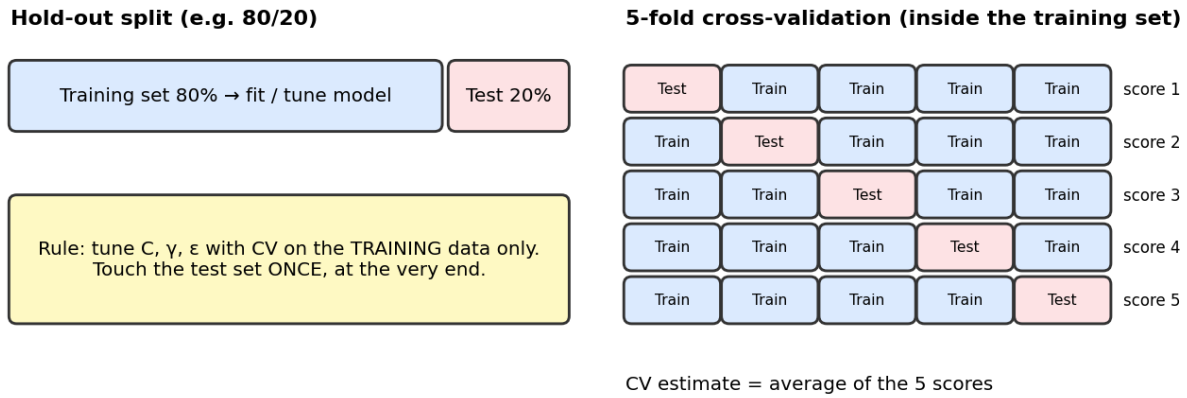
Task	Response (label)	Method in this lecture
Classification	Categorical (e.g. cat / mouse, setosa / versicolor)	**SVM** – Support Vector Machine
Regression	Quantitative / continuous (e.g. mpg, marks)	**SVR** – Support Vector Regression

1.2 Training set, test set and cross-validation

In machine learning we split the data into two parts:

- **Training set** (e.g. a random 70% or 80%) – used to fit (train) the model.
- **Test set** (the remaining 30% or 20%) – used at the end to get predictions. We compare the **predicted** values with the **observed** values to see whether the model is good.

To check whether a line/curve actually works (and to choose between models) we need **hold-out validation** or **k-fold cross-validation**. Without a separate validation/test set we would **overfit** the data.



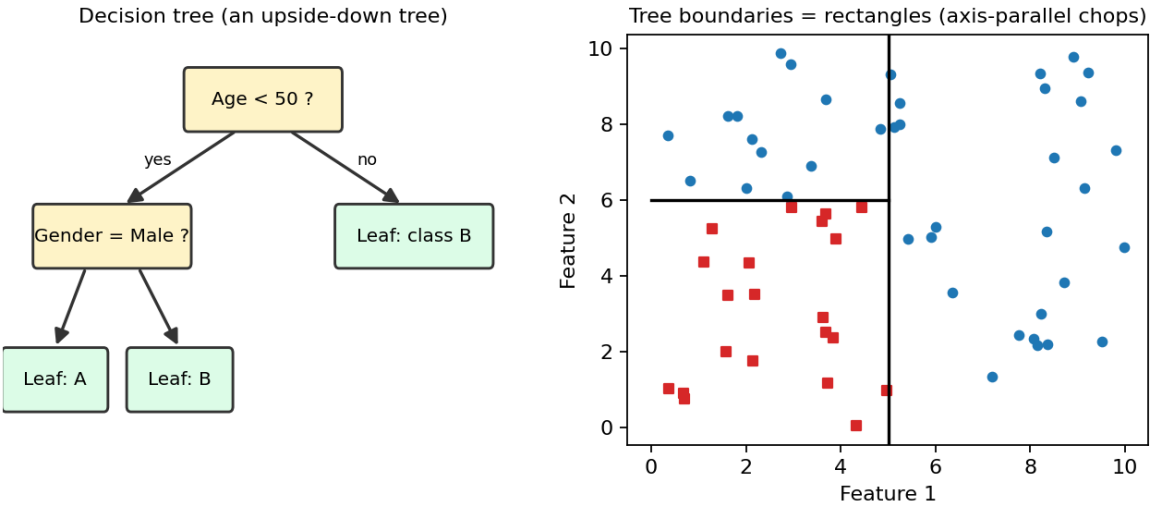
Hold-out split vs 5-fold cross-validation

k-fold cross-validation (e.g. k = 5): break the data into 5 parts. In round 1, part 1 is the test part and the other 4 are for training; in round 2, part 2 is the test part, and so on. Do this 5 times and take the **average** score. Both methods work, but k-fold CV is **more robust** (it does not depend on one lucky split with "easy examples").

1.3 Other ways to create predictive models ("draw lines")

Every classifier is really a way of drawing a line (or curve) that separates classes. We have seen:

- **Decision trees** – think of a tree turned upside down. At each branch there is a condition (e.g. *Age < 50?* then *Gender = Male?*). The tree "chops" the feature space; the chops are feature splits that minimise error. Boundaries are **rectangles**.
- **k-nearest neighbours (KNN)** – look at the k closest points in the training data and take the majority class.
- **Random forests** – many decision trees together (covered earlier).



A decision tree and the rectangular decision boundaries it creates

The "line drawing contest" (from the slides): the same data go into several methods, each tuned over a grid of its hyper-parameter (KNN: k = 5 to 10; tree: cp = 0.01 to 0.10; SVM: C = 0.001 to 10). We pick the one with the best 5-fold CV accuracy (in the slide example: KNN best k = 3, 95%; tree best cp = 0.02, 98%; SVM best C = 9, 100%).

Why are lines a big deal? The true underlying pattern is unknown; there are infinitely many possible lines; with many features we cannot draw them by hand; and simulating intelligent behaviour needs very complex lines.

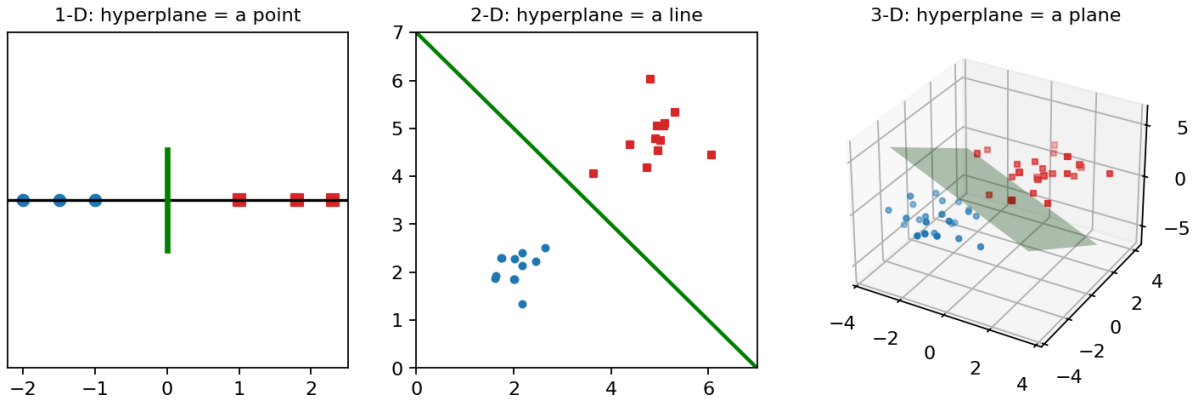
1.4 A bit of history (for completeness)

- **Deep Blue (1997)** – IBM's computer beat world chess champion Garry Kasparov. It did not use data mining, but it was still cool.
- **AlphaGo** – Google's program for the board game **Go** (like chess, but far more possible moves). It beat a top human player – huge news. AlphaGo was just a very complex predictive model that predicted the next move. It has to think many steps ahead, evaluate all possibilities and optimise (keep your pieces, capture the opponent's). It took months on a supercomputer to build.

Data visualisation strategies seen so far: scatter plots (points on a plane), line plots (numeric vs numeric, e.g. time series), bar graphs (categorical vs numeric), histograms (distribution of a continuous variable), heat maps (categorical vs categorical).

2. Hyperplanes and linear classifiers

The idea of an SVM: if we can draw a line that separates two classes, we can classify new points easily. In general this separating object is called a **hyperplane**:



In 1-D a hyperplane is a point, in 2-D a line, in 3-D a plane ("plane" as in a flat surface, not an aeroplane!)

An observation has a predictor vector $x = (x_1, x_2, \ldots, x_p)^T$. A hyperplane in p-dimensional space is

$$\mathbf{w^T x + b = 0}$$

where $w = (w_1, \ldots, w_p)^T$ is **normal** (perpendicular) to the hyperplane and b is an intercept. (Compare with a line $y = mx + c$: we need a **slope** and an **intercept**.)

For binary classification, code the classes as $y_i \in \{-1, +1\}$. The decision rule is:

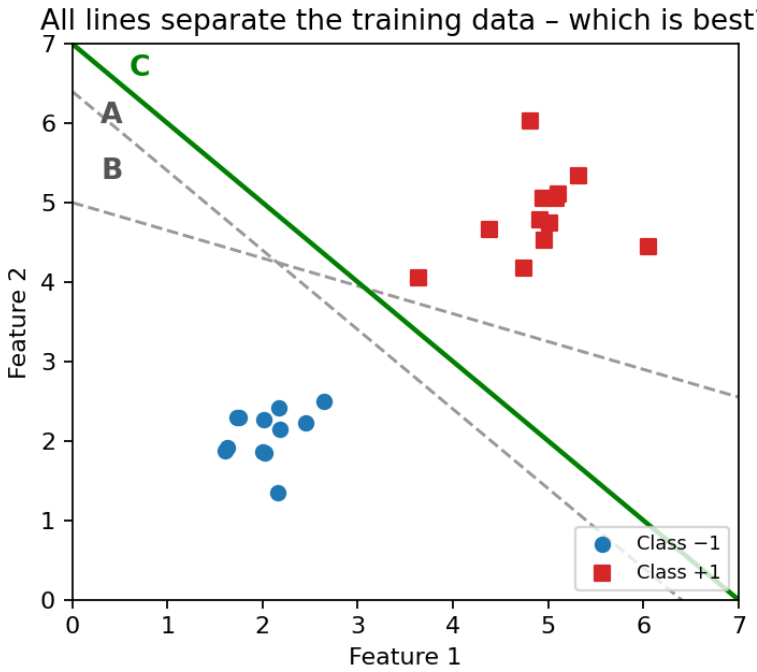
$$\hat{y}(x) = +1 \text{ if } w^T x + b > 0, \; \hat{y}(x) = -1 \text{ if } w^T x + b < 0$$

The size $|w^T x + b|$ is related to how far x is from the hyperplane, but it is **not** the perpendicular distance. The **signed perpendicular distance** is

$$(w^T x + b) / \|w\|$$

3. Which line is best? The maximum-margin idea

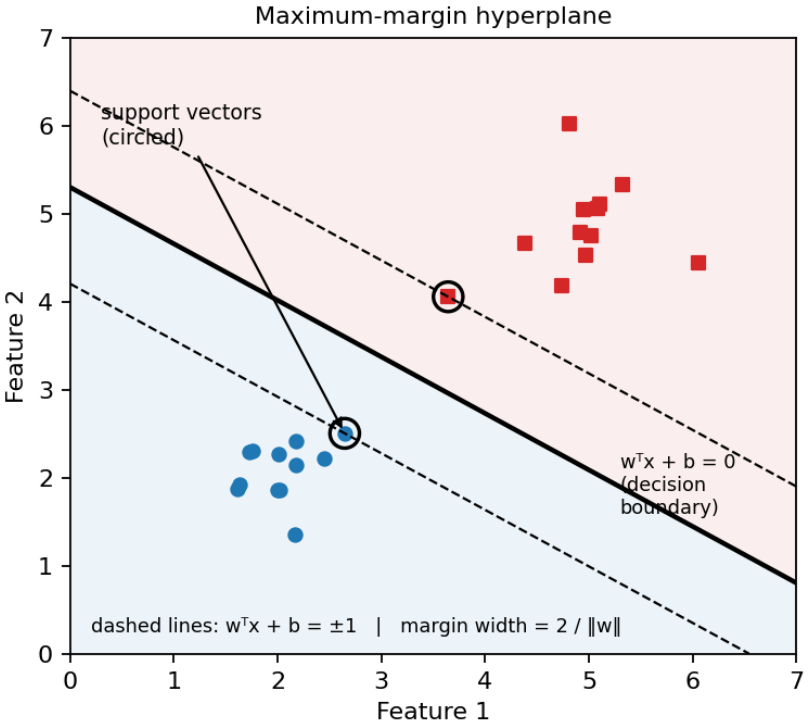
Class question (MCQ-style, like the end-of-semester exam): all three lines A, B and C below separate the training data correctly. Which one is the best?



Answer: the line in the middle (C). A line very close to one class works on the training data, but it is **not robust**. When new test points arrive, a point slightly beyond the old ones may be on the wrong side. A line right in the middle leaves the most room on both sides, so it classifies more new points correctly.

3.1 The two steps of an SVM

1. **Identify the "outside" points** – the points of each class that are closest to the other class. These are called the **support vectors**.
2. **Draw a line equidistant** (same distance) between both sets of support vectors – the line with the **maximum margin** (largest gap) between the classes.



Support vectors (circled) decide the line. Solid line = decision boundary; dashed lines = margin boundaries.

- This is called the **maximum-margin hyperplane**.
- Moving a **non-support vector** a little usually does not change the line; moving a **support vector** can change it.
- To find this middle-ground line we need the right **slope and intercept** – an optimisation task of **maximising the margin** between support vectors. The idea is easy, but there is beautiful maths behind it (next section).

Final exam note (from the slides)

"I'll ask a question related to **how/why** the SVM draws this line." Answer: it finds the support vectors and draws the line equidistant from them with the largest margin, because a large margin gives a stable boundary that generalises better to unseen data.

4. Hard-margin SVM (the maths, simply)

This is an applied course, so you are not expected to derive the maths, but you should know what is happening. (Courses at MIT/Harvard start exactly here: "the equation of the hyperplane is $w^T x + b = 0$".)

Assume the data are **perfectly linearly separable**. Multiplying w and b by the same positive number gives the same line, so we choose the scale so that the closest points satisfy $y_i(w^T x_i + b) = 1$. Then every point must satisfy

$$y_i(w^T x_i + b) \geq 1$$

The two margin lines are $w^T x + b = 1$ and $w^T x + b = -1$. The distance between them (the margin width) is

$$\mathbf{2 / \|w\|}$$

So maximising the margin = minimising $\|w\|$ = (more conveniently) minimising $\frac{1}{2}\|w\|^2$. The hard-margin problem:

$$\text{minimise } \tfrac{1}{2}\|w\|^2 \text{ subject to } y_i(w^T x_i + b) \geq 1,\ i = 1, \ldots, n$$

This is a **convex quadratic** optimisation problem, so any local optimum is also the global optimum (no risk of getting stuck).

Functional vs geometric margin

Name	Formula	Note
Functional margin	$y_i(w^T x_i + b)$	Depends on the arbitrary scale of w and b
Geometric margin	$y_i(w^T x_i + b) / \|w\|$	The real distance – does not depend on scale

Limit of the hard margin

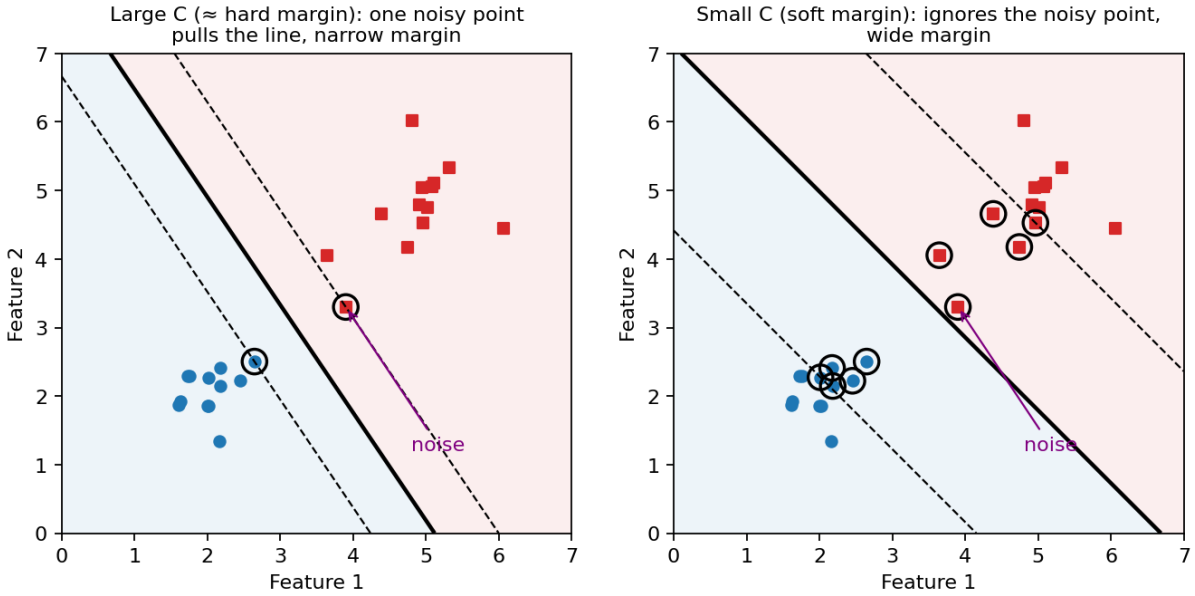
If even **one** observation prevents perfect separation, the constraints cannot all be satisfied (infeasible). Real data have overlap, noise and outliers. This motivates the **soft margin**.

5. Noisy data, soft margin and the cost C

5.1 Noisy observations can ruin the line

Example from the slides: cats and mice. What if there is a **deformed cat** that looks more like a mouse? If this noisy point is chosen as a **support vector**, the line moves towards it.

- A **single noisy example** messes up everything.
- The support vector is chosen incorrectly.
- The model is invalid and has **overfit** – it will not generalise well to real-world cases.



The same data with one noisy red point. Left: a large C chases the noise. Right: a small C ignores it and keeps a wide margin.

5.2 Slack variables

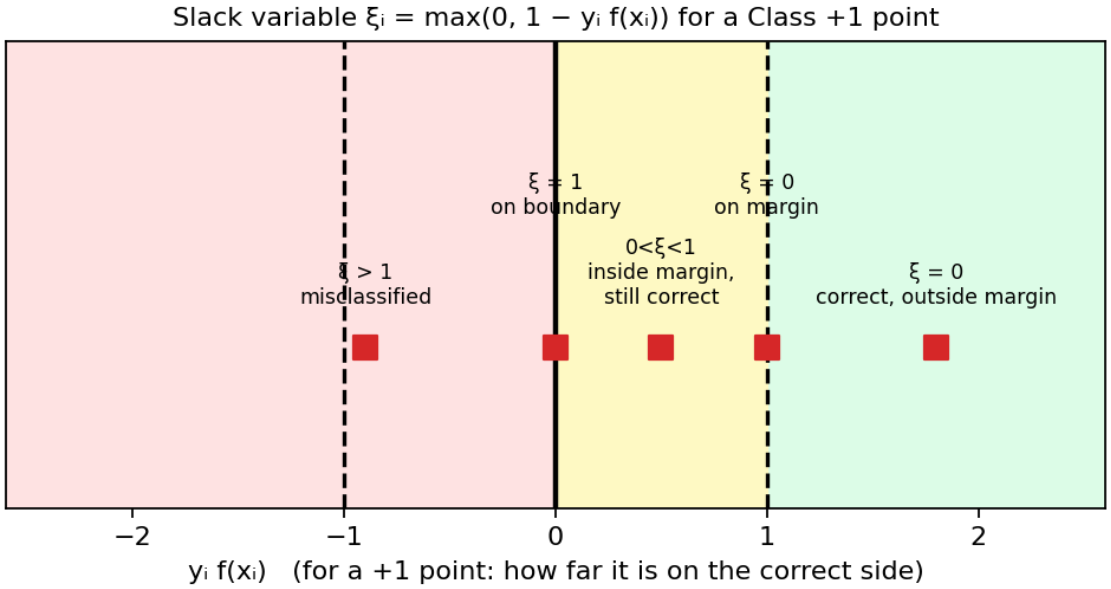
To allow some mistakes, give each observation a non-negative **slack variable** ξ_i ("xi"):

$$y_i(w^T x_i + b) \geq 1 - \xi_i,\ \xi_i \geq 0$$

The **soft-margin** problem is

$$\text{minimise } \tfrac{1}{2}\|w\|^2 + \mathbf{C}\ \Sigma\ \xi_i \text{ subject to the constraints above}$$

$C > 0$ is the **cost** (regularisation) parameter.



Where a Class +1 point sits, and its slack value

Value of ξ_i	Meaning
$\xi_i = 0$	Correct side of the margin (possibly exactly on the margin line)
$0 < \xi_i < 1$	Correctly classified but **inside** the margin
$\xi_i = 1$	Exactly on the decision boundary
$\xi_i > 1$	**Misclassified**

5.3 The cost parameter C

Like **k** in KNN, or **cp** and **max depth** in decision trees, SVM has a **hyper-parameter** that controls model complexity – i.e. how tolerant it is to noisy observations. This is the **cost C**: the "cost" of misclassifying a training point when choosing the support vectors. It sets how resistant the chosen support vectors are to noise.

- **Large C** (e.g. C = 1,000): mistakes on the training data are very costly → "we can't make mistakes" → **narrow margin**, tries to classify every training point correctly. More sensitive to outliers → higher variance → risk of **overfitting**.
- **Small C** (e.g. C = 0.001): mistakes are cheap → "we can make some mistakes" → **wide margin**, more regularised. Training error may be higher, but it often generalises better. Too small → **underfitting**.
- Real-life meaning of "cost": telling a patient he has dengue (or cancer) when he does not is a very costly mistake.
- C controls a **bias–variance trade-off**. We try a bunch of C values (using cross-validation) and choose the one that avoids both overfitting and underfitting.

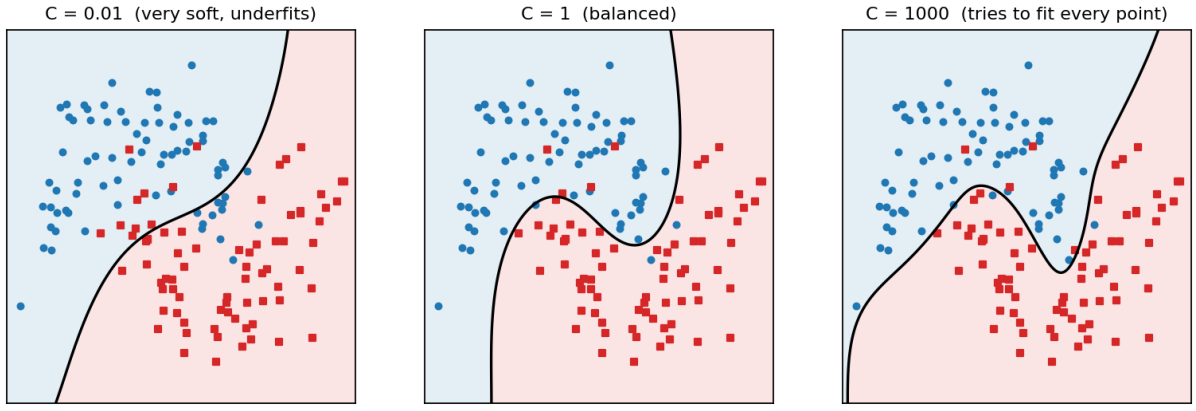
Careful – a statement on the slides is the wrong way round

The slides (and the reading in class) say: "Higher C … too high means we'll *underfit*. Lower C … too low means we'll *overfit*." In standard SVM theory it is the **opposite**:

• **Too high C → overfit** (narrow margin, follows noise, like the left plot above).

• **Too low C → underfit** (very wide margin, too many violations allowed).

The course's written note (section 4.2) and Exercise 3 agree with this standard version – use it in the exam.



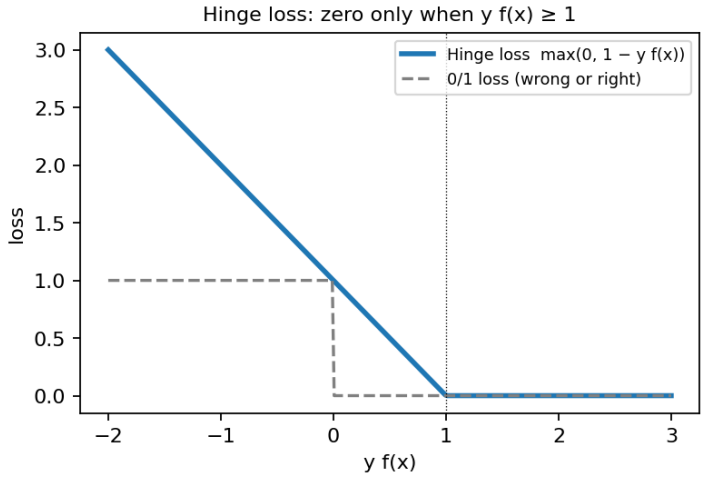
Effect of C with an RBF kernel (γ fixed): small C is too simple; very large C bends around individual points

5.4 Hinge loss

The soft-margin problem can also be written with the **hinge loss** $L_i = \max\{0, 1 - y_i f(x_i)\}$, where $f(x) = w^T x + b$:

$$\text{minimise } \tfrac{1}{2}\|w\|^2 + C \sum \max\{0, 1 - y_i f(x_i)\}$$

The first term likes a **wide margin**; the second term **penalises** points that are on or beyond the wrong side of their margin line.



6. The dual problem and support vectors

Using Lagrange multipliers α_i, the soft-margin SVM has a **dual form**:

$$\text{maximise } \sum \alpha_i - \tfrac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j \mathbf{x_i}^T \mathbf{x_j} \text{ subject to } 0 \le \alpha_i \le C, \sum \alpha_i y_i = 0$$

The fitted coefficients are $w = \sum \alpha_i y_i x_i$. **Only observations with $\alpha_i > 0$ contribute – these are the support vectors.** The decision function is

$$f(x) = \sum_{i \in S} \alpha_i y_i x_i^T x + b \text{ (S = set of support vectors)}$$

Why this matters

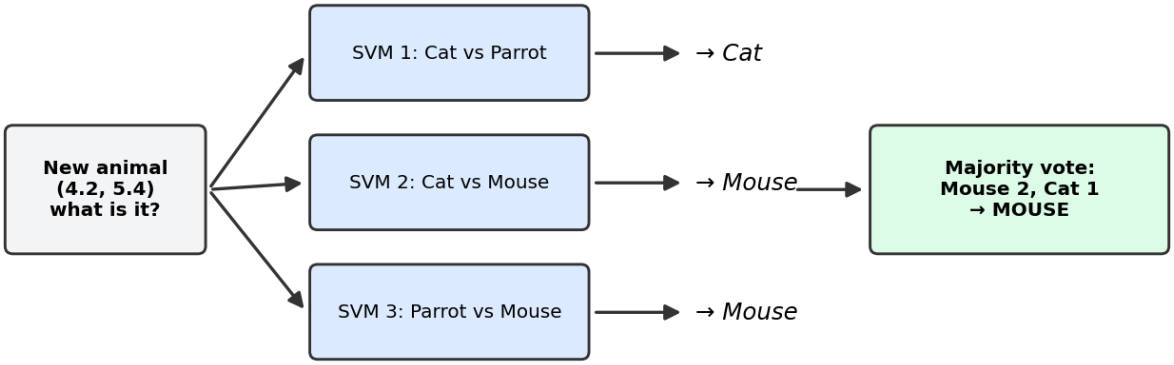
The data appear **only through inner products** $x_i^T x_j$. If we replace these inner products with a **kernel** $K(x_i, x_j)$, we get non-linear boundaries for free. This is the door to the kernel trick (Part 8).

7. Multi-class SVM: the one-vs-one trick

What if we have more than two classes – e.g. **cats, mice and parrots**? How do we draw the line now? KNN and decision trees handle many classes naturally. But an SVM separates only **two** classes, so we cannot use it by itself. We use a trick.

7.1 One-vs-one (OvO)

1. Train one SVM for **every pair** of classes: Cat vs Parrot, Cat vs Mouse, Mouse vs Parrot (run SVM three times → three **sub-models**).
2. When a new (unknown) observation comes in, evaluate it with **each** classifier.
3. Take a **majority vote** as the final prediction.



Worked example from the slides: the new animal at (4.2, 5.4)

Worked example: new point (4.2, 5.4). Cat vs Parrot says **Cat**; Cat vs Mouse says **Mouse**; Parrot vs Mouse says **Mouse**. Votes: Mouse 2, Cat 1 → prediction = **Mouse**.

- For K classes, OvO needs **$K(K-1)/2$** binary classifiers (K = 3 → 3; K = 4 → 6).
- **One-vs-rest (OvR)** is another option: one classifier per class (that class vs all others).
- "This is sort of like cheating, but SVMs cannot handle multi-class by themselves."
- **R does this trick for us in the background:** e1071::svm() uses one-vs-one automatically when the response has more than two levels.

Possible exam task

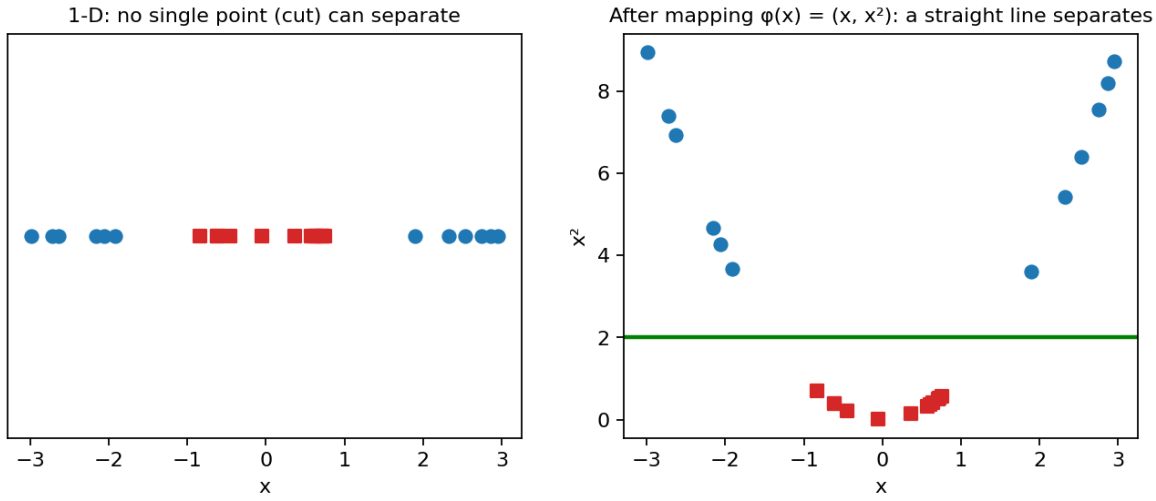
You may be given a dataset and asked to train the model **two classes at a time**: extract two classes, build each SVM, then combine with a majority vote.

8. Non-linear SVM and the kernel trick

A plain SVM finds a **linear** boundary. What if the true boundary is **curved**? Then a straight line does a terrible job – it can be even worse than tossing a coin (a random classifier gives about 50% for two balanced classes). Are we stuck? No.

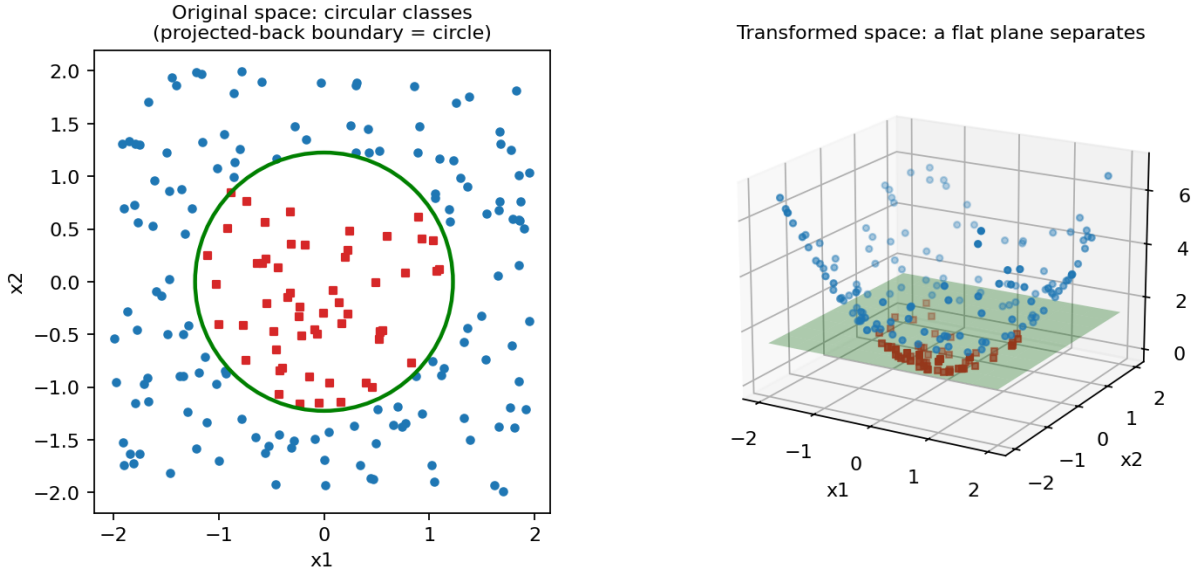
8.1 Idea: transform the data

We **pre-process** (transform / project) the data with a non-linear function $\varphi(x)$ into a richer space. A straight separator (hyperplane) there becomes a **curved** boundary in the original space.



1-D example: after adding x^2 as a new feature, one straight line separates the classes

The video shown in class ("SVM with a polynomial kernel"): red and blue points in a plane cannot be separated by a line. After a polynomial transformation the points are lifted onto a curved (paraboloid) surface in 3-D; now a flat **plane** cuts the two colours apart. Projecting that plane back down gives a **circle** in the original plane. We are not changing the data – only looking at it in a different space.



Circular classes become separable by a flat plane after mapping to $z = x1^2 + x2^2$

8.2 Non-linear SVM – now four steps

1. **Pre-process** the data with a non-linear (kernel) function.
2. Identify the support vectors (in the transformed space).
3. Draw the equidistant line/hyperplane (in the transformed space).
4. **Project the boundary back** onto the original space – nobody can interpret results in the transformed space, so we come back to the original variables. We get a **non-linear decision boundary**.

8.3 What is a kernel? (class activity)

Context	Meaning
Non-data-science	The **core** of something. In computing, the kernel is the core component of an operating system that acts as a bridge between hardware and software layers. (Also: the seed/inner part of a nut or grain.)
Data science / ML	A **kernel function** K(x, z) measures similarity between two observations. It computes the inner product of the transformed vectors **without** actually building them: $K(x, z) = \varphi(x)^T\varphi(z)$.

Building a huge (even infinite) feature map directly would be very expensive. The **kernel trick** gets the same result cheaply. The non-linear decision function is

$$f(x) = \Sigma_{i \in S}\, \alpha_i\, y_i\, \mathbf{K(x_i, x)} + b$$

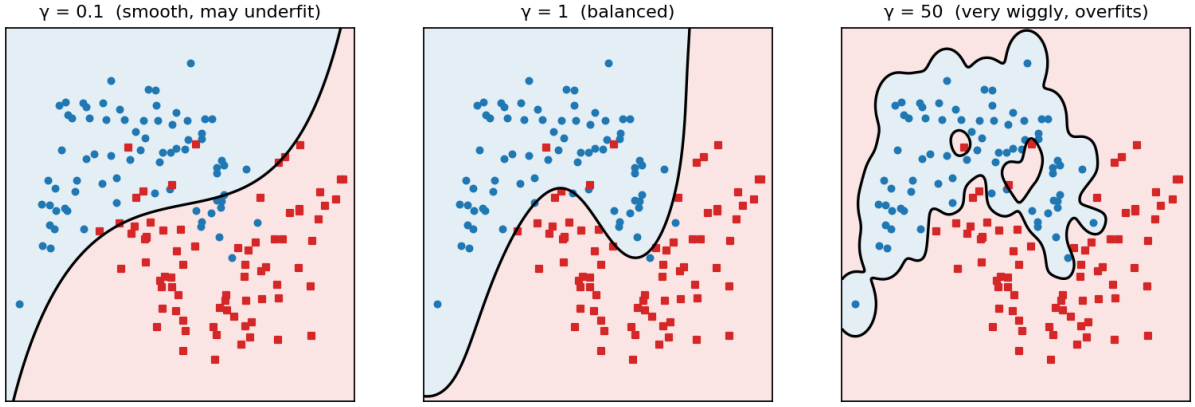
Example map: $\varphi(x_1, x_2) = (x_1, x_2, x_1^2, x_2^2, x_1x_2)$ allows curved boundaries.

8.4 Common kernels

Kernel	Formula	R (e1071)	When to use
Linear	$K(x, z) = x^Tz$	kernel = "linear"	Linear boundary is plausible; already many predictors (e.g. text)
Polynomial	$K(x, z) = (\gamma\, x^Tz + r)^d$	kernel = "polynomial" (degree, gamma, coef0)	When interaction order has meaning; higher d = more curved
Radial basis (RBF / Gaussian)	$K(x, z) = \exp(-\gamma\|x - z\|^2)$	kernel = "radial" (default)	Good general-purpose non-linear choice; local similarity
Sigmoid	$K(x, z) = \tanh(\gamma\, x^Tz + r)$	kernel = "sigmoid"	Like a neural-network activation; not always a valid kernel

The γ (gamma) parameter of the RBF kernel

- **Small γ**: similarity falls off slowly → each support vector influences a wide area → **smooth, less flexible** boundary.
- **Large γ**: similarity falls off quickly → each support vector influences only a small neighbourhood → **local, wiggly** boundary (can overfit).
- **C and γ should be tuned together**.



Effect of γ with an RBF kernel (C fixed)

8.5 What makes a valid kernel?

For inputs $x_1, \ldots, x_n$, the **Gram matrix** K has entries $K_{ij} = K(x_i, x_j)$. A standard sufficient condition: K is **symmetric** and **positive semi-definite**, i.e. $c^T K c \geq 0$ for every real vector c. Then the kernel really is an inner product in some feature space. (See Exercise 4.)

8.6 Choosing a kernel

- Start with a **linear** kernel as a baseline.
- **RBF** is a useful general-purpose non-linear choice (and the natural choice for non-linear regression).
- **Polynomial** when the interaction order means something.
- Choose by **resampling (CV) performance**, never by training accuracy alone.

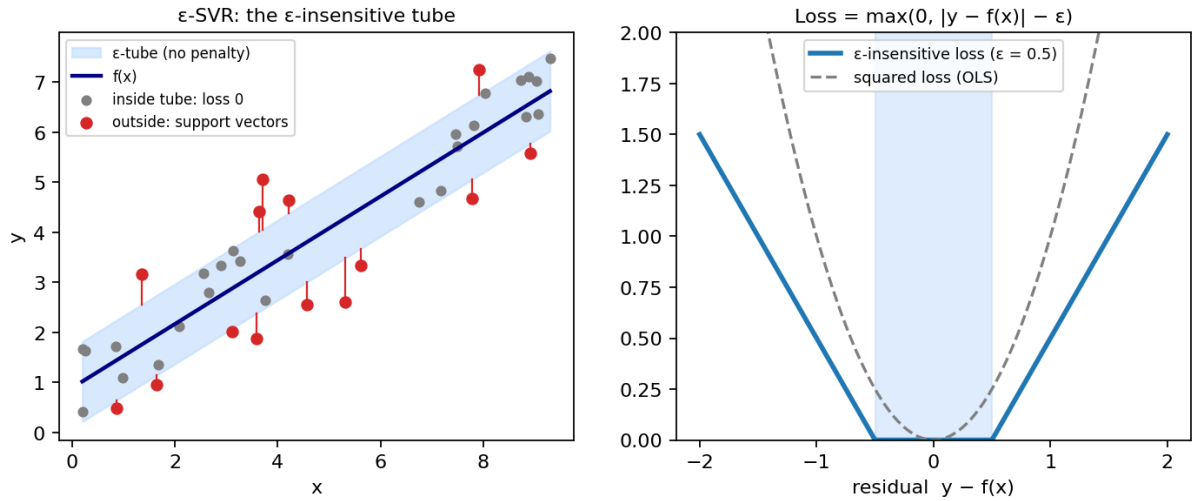
9. Support Vector Regression (ε-SVR)

Classification predicts **categories**; regression predicts **numbers**. SVR follows basically the same procedure:

1. **Identify the support vectors** – but now they are the "outside points" of the **entire** data (there are no classes to separate).
2. **Draw the "middle" line** – the optimal hyperplane (regression function) that maximises the distance between the support vectors. (A few more steps if a kernel is used.)

9.1 The ε-insensitive tube

ε-SVR looks for a function $f(x) = w^T x + b$ that is **as flat as possible**, while allowing errors up to ε **without any penalty**. The region $|y_i - f(x_i)| \leq \varepsilon$ is the **ε-insensitive tube**.



Left: points inside the tube cost nothing; points outside become support vectors. Right: the ε-insensitive loss vs the usual squared loss.

$$L_\varepsilon(y, f(x)) = \max\{0, |y - f(x)| - \varepsilon\}$$

Errors inside the tube have zero loss. Only points outside it affect the loss and may become support vectors.

9.2 Primal formulation

Use two slack variables: ξ_i (point above the tube) and ξ_i^* (below the tube):

$$\text{minimise } \tfrac{1}{2}\|w\|^2 + C \sum (\xi_i + \xi_i^*)$$

$$\text{subject to } y_i - (w^T x_i + b) \le \varepsilon + \xi_i,\ (w^T x_i + b) - y_i \le \varepsilon + \xi_i^*,\ \xi_i, \xi_i^* \ge 0$$

Kernelised prediction: $f(x) = \sum (\alpha_i - \alpha_i^*) K(x_i, x) + b$. Only points with $\alpha_i - \alpha_i^* \neq 0$ contribute.

9.3 Tuning parameters of SVR

Parameter	Main effect
C	Penalty for errors beyond the tube; large C → tries hard to fit training data
ε (epsilon)	Width of the no-penalty tube; larger ε → fewer support vectors, smoother fit
γ (gamma)	Locality / flexibility of the RBF kernel

Real R check (sine data, RBF, C = 10, γ = 1): as ε grows, the number of support vectors drops:

ε	0.05	0.10	0.20	0.30	0.50
Number of support vectors (out of 165)	141	120	75	37	14

A very small ε does **not** automatically improve prediction – it increases the number of support vectors and the sensitivity to noise.

10. Data preparation, tuning and evaluation

10.1 Scaling

SVMs use **distances and inner products**, so a variable measured on a large scale (e.g. income in rupees) would dominate one on a small scale (e.g. age). So **standardise**:

$$z_{ij} = (x_{ij} - \bar{x}_j) / s_j$$

Compute the mean and sd from the **training data only**, then apply the same transformation to validation/test data. e1071::svm() scales numeric predictors by default (scale = TRUE).

10.2 Tuning without leakage

- Use cross-validation **within the training data** to choose C, γ, ε.
- Keep the **test set untouched** until the final model is fixed.
- For an unbiased estimate of the whole selection process, use **nested cross-validation**.
- Use a **logarithmic grid** (e.g. $C = 2^{-1}, \ldots, 2^5$) because useful values differ by orders of magnitude. Start coarse, then refine.

10.3 Evaluation measures

Classification	Regression
Confusion matrix; accuracy or balanced accuracy; sensitivity and specificity; precision, recall, F1; ROC AUC (if scores/probabilities available)	$\text{RMSE} = \sqrt{(1/n) \sum (y_i - \hat{y}_i)^2}$ $\text{MAE} = (1/n) \sum \lvert y_i - \hat{y}_i \rvert$ $R^2 = 1 - \sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2$

Accuracy can be **misleading for imbalanced classes** (e.g. 95% of cases are "no disease" – predicting "no" always gives 95%). Choose a metric that reflects the real cost of errors.

11. R demo: SVM classification

Package: **e1071** (function svm()). In class the lecturer also loaded **caret** (useful for data splitting, e.g. createDataPartition()). Install them once if needed.

11.1 Binary classification: setosa vs versicolor (linear kernel)

```
# install.packages("e1071")        # once
library(e1071)
set.seed(2026)
iris_bin <- subset(iris, Species != "virginica",
                   select = c(Petal.Length, Petal.Width, Species))
iris_bin$Species <- droplevels(iris_bin$Species)   # remove the empty level

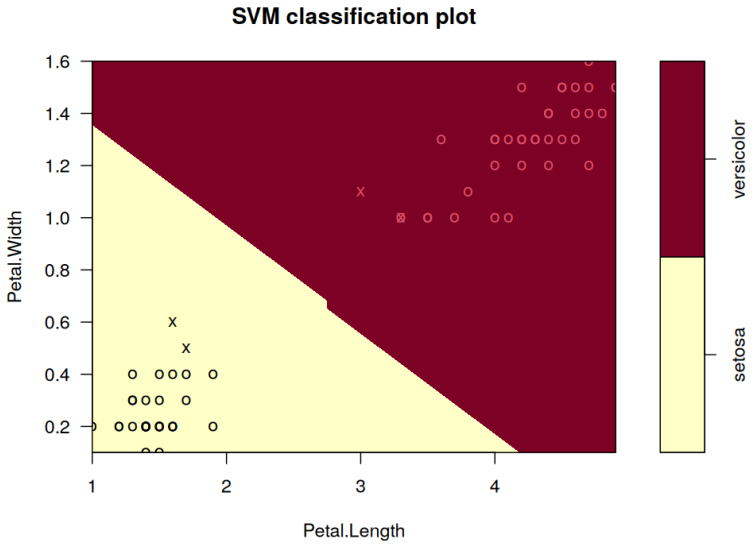
id <- sample(seq_len(nrow(iris_bin)), size = 0.70 * nrow(iris_bin))
train_cls <- iris_bin[id, ]
test_cls  <- iris_bin[-id, ]

fit_linear <- svm(Species ~ ., data = train_cls,
                  kernel = "linear", cost = 1, scale = TRUE)
fit_linear
pred_linear <- predict(fit_linear, newdata = test_cls)
table(Observed = test_cls$Species, Predicted = pred_linear)
mean(pred_linear == test_cls$Species)
```

```
Parameters:
   SVM-Type:  C-classification
 SVM-Kernel:  linear
       cost:  1
Number of Support Vectors:  4

            Predicted
Observed     setosa versicolor
  setosa         17          0
  versicolor      0         13
[1] 1                         # test accuracy = 100%
```

Only **4 support vectors** (out of 70 training flowers) decide the boundary. Setosa and versicolor are easy to separate, so accuracy is 100%.



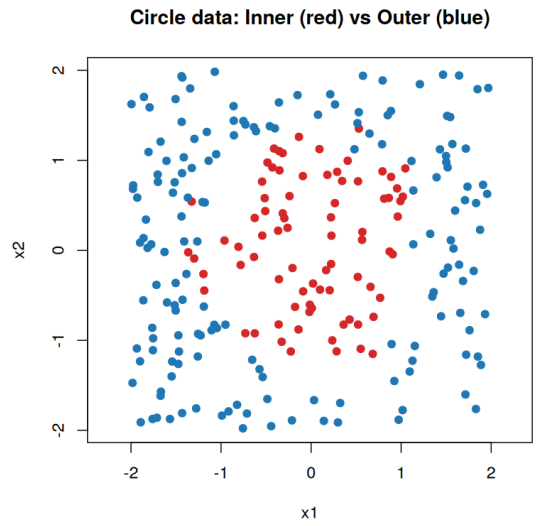
plot(fit_linear, train_cls, Petal.Width ~ Petal.Length). Background colour = predicted class; x = support vectors, o = other points; symbol colour = true class.

11.2 Non-linear data: a circle (linear vs RBF)

```
set.seed(2026)
n <- 240
x1 <- runif(n, -2, 2)
x2 <- runif(n, -2, 2)
radius2 <- x1^2 + x2^2
class_y <- factor(ifelse(radius2 + rnorm(n, 0, 0.30) > 1.7, "Outer", "Inner"))
circle_dat <- data.frame(x1, x2, class_y)
id <- sample(seq_len(n), size = 0.75 * n)
train_circle <- circle_dat[id, ];  test_circle <- circle_dat[-id, ]

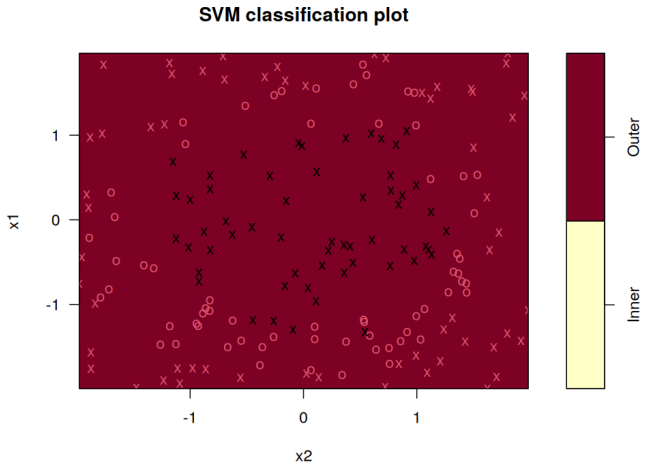
fit_circle_linear <- svm(class_y ~ ., data = train_circle, kernel = "linear", cost = 1)
fit_circle_rbf    <- svm(class_y ~ ., data = train_circle, kernel = "radial",
                         cost = 10, gamma = 1)
pred_lin <- predict(fit_circle_linear, test_circle)
pred_rbf <- predict(fit_circle_rbf, test_circle)
c(linear_accuracy = mean(pred_lin == test_circle$class_y),
  rbf_accuracy    = mean(pred_rbf == test_circle$class_y))
plot(fit_circle_rbf, train_circle)
```

```
linear_accuracy    rbf_accuracy
      0.6166667       0.9666667
# support vectors: linear 115, RBF 33
```

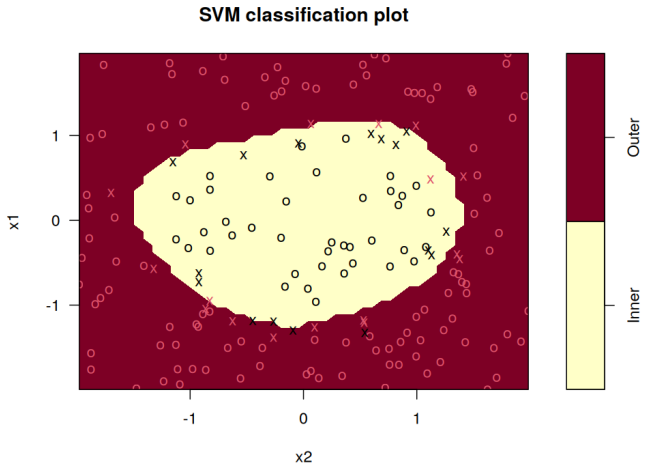


The circle data: Inner (red) inside, Outer (blue) outside

The linear SVM gets only **61.7%** – exactly what you get by always guessing "Outer" (37 of the 60 test points are Outer). So the straight line is useless here. It also needs 115 support vectors because it cannot find a good line. The RBF kernel gets **96.7%** with only 33 support vectors.



Linear kernel: a straight boundary cannot capture a circle



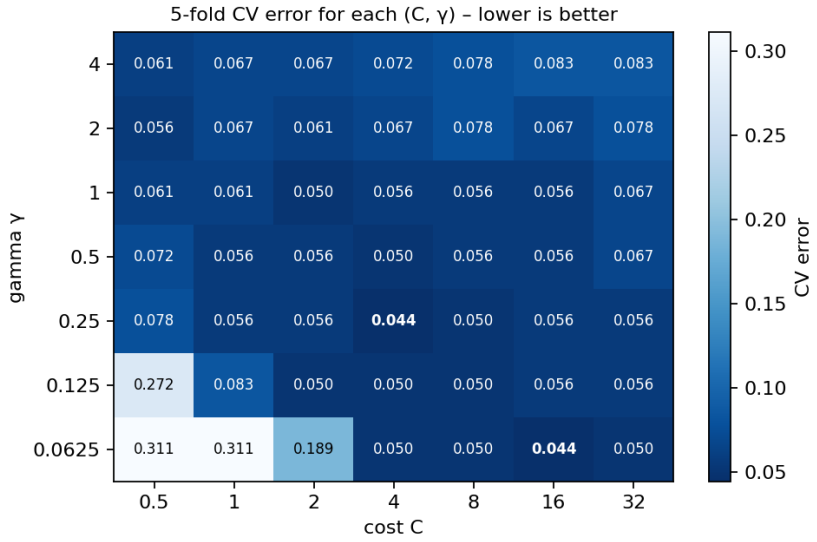
RBF kernel: the boundary is a closed curve around the inner class

11.3 Tuning C and γ with 5-fold CV

```
set.seed(2026)
tuned_cls <- tune(svm, class_y ~ ., data = train_circle, kernel = "radial",
                  ranges = list(cost = 2^(-1:5), gamma = 2^(-4:2)),
                  tunecontrol = tune.control(cross = 5))
summary(tuned_cls)
best_cls  <- tuned_cls$best.model
pred_best <- predict(best_cls, newdata = test_circle)
table(Observed = test_circle$class_y, Predicted = pred_best)
mean(pred_best == test_circle$class_y)
```

```
- best parameters:
 cost  gamma
   16 0.0625
- best performance: 0.04444444        # CV error = 4.4%

        Predicted
Observed Inner Outer
   Inner    21     2
   Outer     0    37
[1] 0.9666667
```



CV error for all 49 combinations (7 costs × 7 gammas). The grid is logarithmic.

tune() tried 7 × 7 = 49 combinations, each with 5-fold CV. The best had CV error 4.4% (several combinations are almost as good). The test accuracy of the tuned model is 96.7%.

11.4 Decision values and probabilities

SVM output is **not** a probability. decision.values = TRUE gives signed scores (distance-like). probability = TRUE makes e1071 fit an extra **calibration** step to estimate probabilities.

```
fit_prob <- svm(class_y ~ ., data = train_circle, kernel = "radial",
                cost = 10, gamma = 1, probability = TRUE)
pred_prob <- predict(fit_prob, test_circle, probability = TRUE)
head(attr(pred_prob, "probabilities"))
```

```
         Outer       Inner
1 0.9954401733 0.004559827
2 0.0002168569 0.999783143
3 0.9961027888 0.003897211
...
# decision values (Outer/Inner): 3.11, -4.93, 3.20, ...  (positive -> Outer)
```

If well-calibrated risk estimates matter (e.g. medical risk), check the calibration itself.

11.5 Class weights (imbalanced classes)

```
fit_weighted <- svm(class_y ~ ., data = train_circle, kernel = "radial",
                    cost = 10, gamma = 1,
                    class.weights = c(Inner = 2, Outer = 1))
```

Mistakes on the "Inner" class now cost twice as much. In our run, test errors on Inner dropped from 2 to 1. Weights should reflect the real cost of errors, not be chosen mechanically.

12. R demo: Support Vector Regression

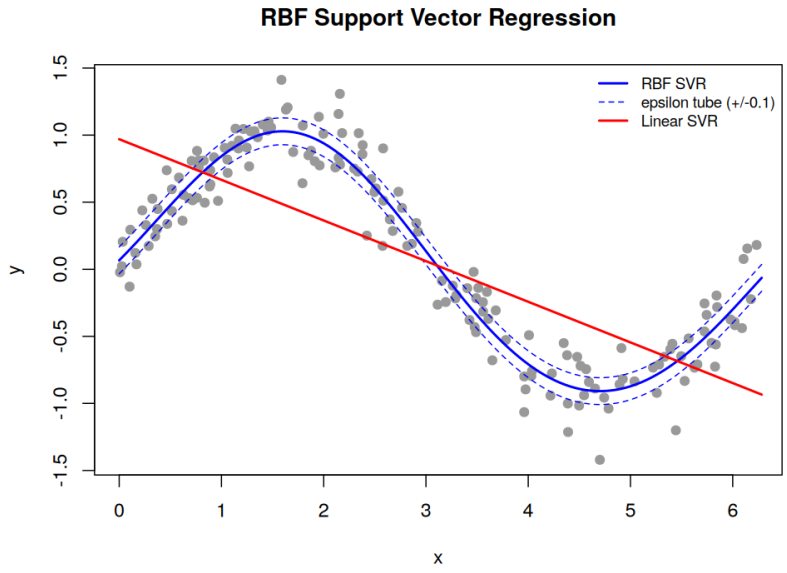
```
library(e1071)
set.seed(2026)
n <- 220
x <- runif(n, 0, 2 * pi)
y <- sin(x) + rnorm(n, mean = 0, sd = 0.18)     # non-linear truth: sine curve
reg_dat <- data.frame(x, y)
id <- sample(seq_len(n), size = 0.75 * n)
train_reg <- reg_dat[id, ];  test_reg <- reg_dat[-id, ]

svr_linear <- svm(y ~ x, data = train_reg, type = "eps-regression",
                  kernel = "linear", cost = 10, epsilon = 0.10)
svr_rbf    <- svm(y ~ x, data = train_reg, type = "eps-regression",
                  kernel = "radial", cost = 10, gamma = 1, epsilon = 0.10)
pred_linear <- predict(svr_linear, test_reg)
pred_rbf    <- predict(svr_rbf, test_reg)

rmse <- function(actual, predicted) sqrt(mean((actual - predicted)^2))
mae  <- function(actual, predicted) mean(abs(actual - predicted))
rbind(Linear_SVR = c(RMSE = rmse(test_reg$y, pred_linear), MAE = mae(test_reg$y, pred_linear)),
      RBF_SVR    = c(RMSE = rmse(test_reg$y, pred_rbf),    MAE = mae(test_reg$y, pred_rbf)))

x_grid <- data.frame(x = seq(0, 2 * pi, length.out = 300))
y_grid <- predict(svr_rbf, newdata = x_grid)
plot(train_reg$x, train_reg$y, pch = 19, col = "grey60",
     xlab = "x", ylab = "y", main = "RBF Support Vector Regression")
lines(x_grid$x, y_grid, col = "blue", lwd = 2)
```

```
               RMSE       MAE
Linear_SVR 0.4139238 0.3566986
RBF_SVR    0.1421773 0.1109824
```



RBF SVR follows the sine curve; the linear SVR can only draw a straight line. (Dashed blue = ε-tube, added for illustration.)

The RBF SVR error is about **one third** of the linear SVR error, because the true relationship is non-linear. Note: the noise sd is 0.18, so an RMSE of 0.14 is about as good as possible.

12.1 Tuning C, γ and ε

```
set.seed(2026)
tuned_reg <- tune(svm, y ~ x, data = train_reg, type = "eps-regression",
                  kernel = "radial",
                  ranges = list(cost = c(0.1, 1, 10, 100),
                                gamma = c(0.05, 0.1, 0.5, 1, 2),
                                epsilon = c(0.05, 0.10, 0.20, 0.30)),
                  tunecontrol = tune.control(cross = 5))
summary(tuned_reg)
best_reg   <- tuned_reg$best.model
pred_tuned <- predict(best_reg, newdata = test_reg)
c(RMSE = rmse(test_reg$y, pred_tuned), MAE = mae(test_reg$y, pred_tuned))
```

```
- best parameters:  cost = 10, gamma = 1, epsilon = 0.3
- best performance (CV MSE): 0.0328
     RMSE       MAE
0.1508785 0.1186085        # only 37 support vectors
```

The tuned model chose a **wider tube** (ε = 0.3) → only 37 support vectors and almost the same test error (slightly higher on this particular test set). A simpler model that is nearly as accurate is usually preferred. For a rigorous comparison, evaluate all candidate models on the **same** resampling splits.

13. In-class exercises

13.1 Exercise A – SVM on iris: versicolor vs virginica

Task: (1) load iris; keep only **versicolor** and **virginica** (remove setosa); (2) fit an SVM of Species on Petal.Length + Petal.Width using default arguments (you may also try kernel = "linear"); (3) split the data (80/20) and test.

```
library(caret)     # useful for splitting (createDataPartition)
library(e1071)
data(iris)
dim(iris)                                              # 150 5

data_filtered <- iris[iris$Species != "setosa", ]   # or subset(iris, Species != "setosa")
data_filtered$Species <- factor(data_filtered$Species)  # drop the unused level
dim(data_filtered)                                     # 100 5
table(data_filtered$Species)

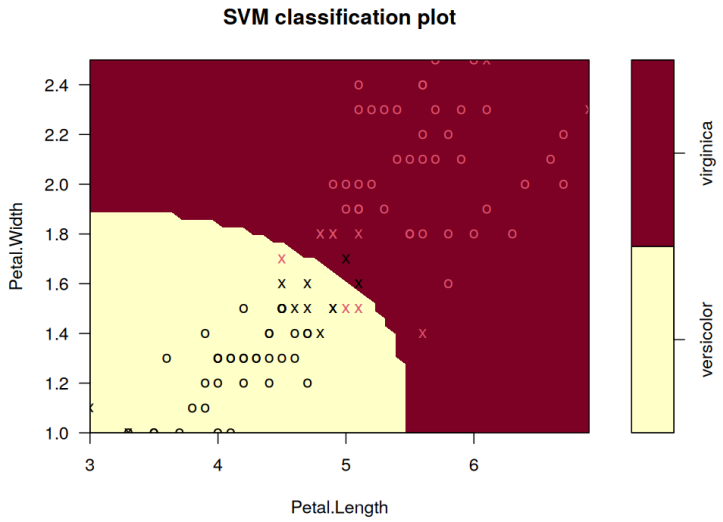
m0 <- svm(Species ~ Petal.Length + Petal.Width, data = data_filtered)  # defaults
m0
```

```
[1] 150   5
[1] 100   5
versicolor  virginica
        50         50
   SVM-Type:  C-classification
 SVM-Kernel:  radial            # default kernel
       cost:  1                 # default cost
Number of Support Vectors:  23
```

Two things the lecturer praised

• Converting Species with factor() after filtering – otherwise the empty "setosa" level remains and svm() may treat it as a 3-class problem.

• Splitting the data (train/test) before modelling. The dimension check (150 → 100) confirms setosa was removed.



Default (radial) SVM for versicolor vs virginica – harder than setosa because the classes overlap

Homework: split first, then compare SVM with logistic regression and random forest

```
library(randomForest)
set.seed(123)
ids   <- sample(1:nrow(data_filtered), 0.8 * nrow(data_filtered), replace = FALSE)
train <- data_filtered[ids, ];  test <- data_filtered[-ids, ]

m_svm <- svm(Species ~ Petal.Length + Petal.Width, data = train, kernel = "linear")
m_log <- glm(Species ~ Petal.Length + Petal.Width, data = train, family = binomial)
m_rf  <- randomForest(Species ~ Petal.Length + Petal.Width, data = train)

p_svm <- predict(m_svm, test)
p_log <- ifelse(predict(m_log, test, type = "response") > 0.5, "virginica", "versicolor")
p_rf  <- predict(m_rf, test)
c(SVM = mean(p_svm == test$Species), Logistic = mean(p_log == test$Species),
  RandomForest = mean(p_rf == test$Species))
```

```
        SVM     Logistic RandomForest
       0.95         0.95         0.95      # 19 of 20 test flowers correct
```

All three give 95% on this small test set (20 flowers) – one mistake each. With so few test cases, use cross-validation to compare them more reliably. (Your numbers may differ with a different seed.)

13.2 Exercise B – mtcars: linear regression vs SVR

Task: use mtcars; predict **mpg** using **wt** and **hp**; (1) build a linear regression model; (2) build an SVR model; (3) compare the RMSE on the test set (80/20 split).

Notes from the svm() help page

- svm() can do general **regression and classification** (and density estimation).
- The default **type** is C-classification. **If you do not change the type, it will do classification!** For regression you need `type = "eps-regression"`. (Actually, if the response is numeric e1071 switches to eps-regression automatically – but always write it explicitly.)
- Kernels available: `"linear", "polynomial", "radial", "sigmoid"`. For non-linear relationships, the **radial** kernel is the natural choice.

```
set.seed(123)
ids <- sample(1:nrow(mtcars), 0.8 * nrow(mtcars), replace = FALSE)  # unique rows
train_data <- mtcars[ids, ]
test_data  <- mtcars[-ids, ]

lm_mtcars  <- lm(mpg ~ wt + hp, data = train_data)
svr_mtcars <- svm(mpg ~ wt + hp, data = train_data,
                  type = "eps-regression", kernel = "radial")

pred_lm  <- predict(lm_mtcars,  newdata = test_data)
pred_svr <- predict(svr_mtcars, newdata = test_data)

rmse <- function(pred, obs) {
  sq_err <- (pred - obs)^2
  sqrt(sum(sq_err) / length(obs))      # <- the LAST line is what R returns
}
rmse(pred_lm,  test_data$mpg)
rmse(pred_svr, test_data$mpg)
```

```
                   actual    lm   svr
Mazda RX4 Wag        21.0 22.80 21.55
Valiant              18.1 20.59 19.89
Merc 450SE           16.4 15.57 13.72
Merc 450SL           17.3 16.96 14.73
Lincoln Continental  10.4  8.88 14.00
Toyota Corona        21.5 24.91 24.45
Pontiac Firebird     19.2 16.66 14.49
 RMSE_lm RMSE_svr
2.090903 2.955185
```

Why sample() with `replace = FALSE`? We need **unique** row IDs; the test set is simply `-ids` (all rows not chosen). Note 0.8 × 32 = 25.6 → 25 training cars and **7 test cars**.

Result: here the linear model has the **lower** RMSE (2.09 vs 2.96). As in class, the SVR is worse in this particular case – possibly because we used only two predictors, the default C/γ/ε were not tuned, and the sample is tiny. (Using all 10 predictors: SVR RMSE 2.79 vs lm 2.59 – still not better.) A more flexible model is **not automatically** better.

The bug in class (a lesson about R functions)

When the lecturer first wrote the RMSE function, it returned the wrong value, because an R function returns the value of its **last evaluated expression**. If the last line is an assignment (e.g. `err <- ...`) or an intermediate step, you don't get the RMSE. Fix: make sure the last line (or an explicit `return()`) is the final RMSE. Also check the function once with a known value (e.g. rmse(c(1,2), c(1,4)) should be √2 ≈ 1.414).

14. Common mistakes and comparison with other methods

14.1 Practical mistakes to avoid

Mistake	Why it is a problem
Not scaling predictors	Large-scale variables dominate distances and inner products
Tuning on the test set	Leaks information; the final performance becomes too optimistic
Choosing parameters by training accuracy	Flexible kernels (large C, large γ) overfit
Assuming a non-linear kernel is always better	A linear SVM can be simpler, faster and more stable
Treating the decision score as a probability	Needs calibration (probability = TRUE)
Ignoring class imbalance	Accuracy hides poor minority-class performance – use class weights and better metrics
Searching a huge grid	Start with a coarse log grid, find a good region, then refine
Thinking SVM is automatically robust to outliers	A large C makes the boundary sensitive to unusual points

14.2 SVM vs other methods

Aspect	SVM	Logistic / linear regression	Tree-based methods
Boundary	Linear or kernel-based non-linear	Usually linear unless features are engineered	Piecewise rectangular, non-linear
Scaling needed	Usually yes	Often advisable	Usually no
Native probability	No	Yes (logistic)	Often gives estimated probabilities
Interpretability	Moderate (linear); lower (kernels)	Relatively high	Varies
High-dimensional data	Often strong	Can be strong with regularisation	Varies
Key tuning	C, kernel parameters (γ, degree), ε for SVR	Penalty and mixing parameter (if regularised)	Depth, node size, number of trees

No method wins on every problem. Choose by predictive performance, interpretability, computation, calibration and the consequences of errors.

When are SVMs especially useful? Many predictors; non-linear relationships; when you want a flexible boundary without building many transformed variables yourself; when prediction matters more than interpreting coefficients.

15. Exercises with worked solutions

Exercise 1 – distance and class prediction

Decision function $f(x) = 2x_1 - x_2 - 3$. (a) State w and b. (b) Classify $x_A = (3, 1)$ and $x_B = (1, 2)$. (c) Find the signed perpendicular distances.

Solution. Compare with $f(x) = w^T x + b$: $\mathbf{w = (2, -1)^T}$, $\mathbf{b = -3}$, and $\|w\| = \sqrt{(2^2 + (-1)^2)} = \sqrt{5} \approx 2.236$.

Point	f(x)	Class (sign)	Signed distance = f(x)/‖w‖
$x_A = (3, 1)$	$2(3) - 1 - 3 = 2$	+1	$2/\sqrt{5} \approx 0.894$
$x_B = (1, 2)$	$2(1) - 2 - 3 = -3$	−1	$-3/\sqrt{5} \approx -1.342$

Exercise 2 – slack variables

Values of $y_i f(x_i)$ are 1.4, 0.6, 0 and −0.3. Find the minimum slack and interpret.

Solution. $\xi_i = \max\{0, 1 - y_i f(x_i)\}$.

$y_i f(x_i)$	ξ_i	Interpretation
1.4	$\max(0, -0.4) = 0$	Correct side, beyond the margin line
0.6	0.4	Correctly classified but inside the margin
0	1	On the decision boundary
−0.3	1.3	Misclassified

Exercise 3 – effects of C and γ (RBF SVM)

- **Increase C (γ fixed):** violations are penalised more → narrower margin, tries harder to classify training cases correctly.
- **Increase γ (C fixed):** each support vector influences a smaller neighbourhood → more irregular, local boundary.
- **Both C and γ very large:** a very complex boundary that follows noise/outliers → low training error but poor generalisation (**overfitting**).
- These are general tendencies, not guarantees – behaviour depends on the data.

Exercise 4 – checking a kernel

$x_1 = 1$, $x_2 = 2$, $K(x, z) = xz + 1$. (a) Gram matrix. (b) Show it is positive semi-definite. (c) Give a feature map.

(a) $K(1,1) = 1 \cdot 1 + 1 = 2$; $K(1,2) = K(2,1) = 1 \cdot 2 + 1 = 3$; $K(2,2) = 2 \cdot 2 + 1 = 5$. So **K = [[2, 3], [3, 5]]**.

(b) Leading principal minors: $2 > 0$ and $\det(K) = 2 \cdot 5 - 3 \cdot 3 = 1 > 0$ → positive definite, hence positive semi-definite. Eigenvalues $\lambda = (7 \pm \sqrt{45})/2 =$ **6.854 and 0.146** (both positive – checked with eigen() in R).

(c) $\varphi(x) = (x, 1)^T$, because $\varphi(x)^T \varphi(z) = x \cdot z + 1 \cdot 1 = xz + 1$.

Exercise 5 – ε-insensitive loss

$\varepsilon = 0.2$; residuals 0.10, −0.35, 0.80. Find the losses, which points are outside the tube, and the total.

Residual r	|r|	Loss = max(0, |r| − 0.2)	Outside tube?
0.10	0.10	0	No
−0.35	0.35	0.15	Yes
0.80	0.80	0.60	Yes

Total loss = 0 + 0.15 + 0.60 = **0.75**.

Exercise 6 – R classification task (all three iris species)

```
library(e1071)
set.seed(2026)
id <- sample(seq_len(nrow(iris)), size = floor(0.70 * nrow(iris)))
train <- iris[id, ];  test <- iris[-id, ]

linear_fit  <- svm(Species ~ ., data = train, kernel = "linear", cost = 1, scale = TRUE)
radial_tune <- tune(svm, Species ~ ., data = train, kernel = "radial",
                    ranges = list(cost = 2^(-1:5), gamma = 2^(-5:1)),
                    tunecontrol = tune.control(cross = 5))
radial_fit  <- radial_tune$best.model

pred_linear <- predict(linear_fit, test);  pred_radial <- predict(radial_fit, test)
table(Observed = test$Species, Predicted = pred_linear)
table(Observed = test$Species, Predicted = pred_radial)
c(linear_accuracy = mean(pred_linear == test$Species),
  radial_accuracy = mean(pred_radial == test$Species))
c(linear_support_vectors = linear_fit$tot.nSV, radial_support_vectors = radial_fit$tot.nSV)
radial_tune$best.parameters
```

```
            Predicted                        (same table for linear and radial)
Observed     setosa versicolor virginica
  setosa         18          0         0
  versicolor      0         18         2
  virginica       0          0         7
linear_accuracy radial_accuracy
      0.9555556       0.9555556
linear_support_vectors radial_support_vectors
                    24                     31
  cost   gamma
     8 0.03125               # best CV error 2.9%
```

Choice: both models get 95.6% (43/45) and make the same two mistakes (versicolor predicted as virginica). The test set is small, so one or two flowers change the accuracy a lot. Since they perform the same, the **linear model** is a defensible choice: it is simpler and uses fewer support vectors (24 vs 31). Behind the scenes, 3 species → 3 one-vs-one sub-models.

Exercise 7 – R regression task (mtcars, 4 predictors)

```
set.seed(2026)
dat <- mtcars[, c("mpg", "wt", "hp", "disp", "qsec")]
id  <- sample(seq_len(nrow(dat)), size = floor(0.75 * nrow(dat)))
train <- dat[id, ];  test <- dat[-id, ]

linear_fit <- svm(mpg ~ ., data = train, type = "eps-regression",
                  kernel = "linear", cost = 1, epsilon = 0.1, scale = TRUE)
radial_tune <- tune(svm, mpg ~ ., data = train, type = "eps-regression", kernel = "radial",
                    ranges = list(cost = c(0.1, 1, 10, 100), gamma = c(0.01, 0.1, 0.5, 1),
                                  epsilon = c(0.05, 0.1, 0.2, 0.5)),
                    tunecontrol = tune.control(cross = 5))
radial_fit <- radial_tune$best.model
pred_linear <- predict(linear_fit, test);  pred_radial <- predict(radial_fit, test)
rmse <- function(y, p) sqrt(mean((y - p)^2));  mae <- function(y, p) mean(abs(y - p))
rbind(Linear = c(RMSE = rmse(test$mpg, pred_linear), MAE = mae(test$mpg, pred_linear)),
      RBF    = c(RMSE = rmse(test$mpg, pred_radial), MAE = mae(test$mpg, pred_radial)))
radial_tune$best.parameters
```

```
           RMSE      MAE
Linear 3.924017 3.012459
RBF    4.137543 3.583697
   cost gamma epsilon
     10  0.01     0.5
```

Discussion: with only 32 cars, the test set has just **8** cars. The comparison is unstable – a different split could reverse the ranking. Here the linear SVR is slightly better. Repeated or nested cross-validation would be more reliable; an independent test set would be better if more data were available.

16. Introduction to Neural Networks

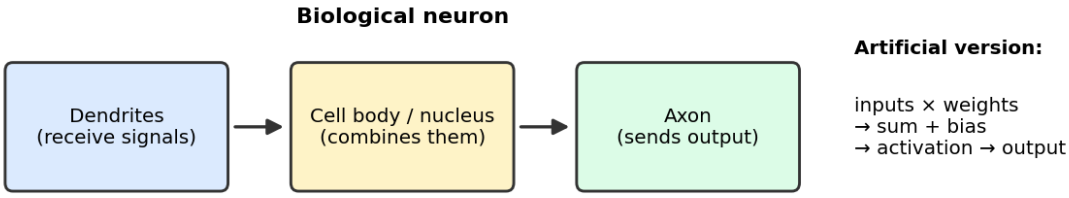
Syllabus position: unsupervised learning (clustering, dimension reduction, recommender systems) → supervised learning (parametric and non-parametric algorithms, **SVM and kernels**) → what is left: **neural networks**, then deep learning. From now on there will be a lot of coding.

16.1 Class activity: main components of a neural network

Answer given in class: **input layer, hidden layer(s) and output layer**. We add: **weights**, **biases**, **activation functions**, a **loss function** and an **optimiser** (training method).

Reference book: **C. C. Aggarwal, *Neural Networks and Deep Learning: A Textbook***. Chapters include: introduction to neural networks; machine learning with shallow neural networks; training deep neural networks; radial basis function networks; restricted Boltzmann machines; recurrent neural networks; convolutional neural networks; deep reinforcement learning; advanced topics in deep learning.

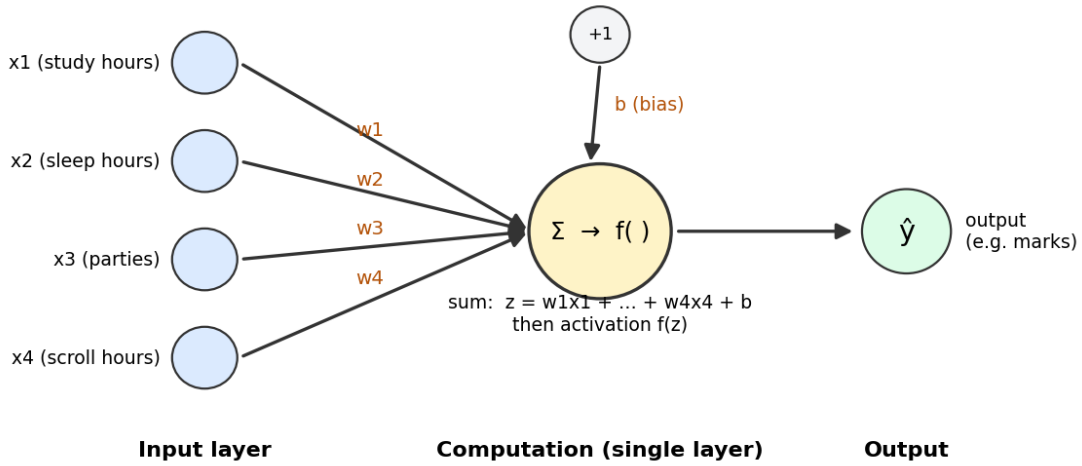
16.2 From biological neuron to artificial neuron



In our brain, a **neuron** receives signals through its **dendrites**, combines them in the cell body (nucleus), and sends an output through its **axon**. Scientists copied this idea into a mathematical model: an **artificial neural network (ANN)**, made of computational units also called **neurons**. They are not biological – they are artificially created – but loosely we just say "neural network" or "neural net".

16.3 The perceptron (single-layer network)

In a **single-layer network**, a set of inputs is mapped directly to an output using a generalised version of a linear function. This simplest neural network is called the **perceptron** – the basic unit of an ANN. (If asked to draw a perceptron, you should be able to draw this.)



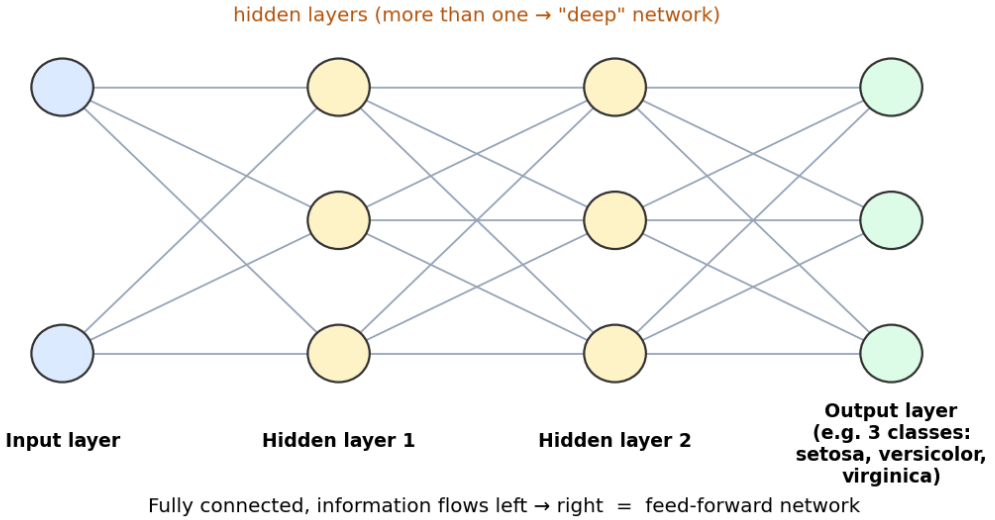
Class example – group-project marks: suppose marks depend on hours studied (x_1), hours slept (x_2), number of parties (x_3) and hours scrolling social media (x_4). Some inputs help (positive weight), others hurt (negative weight), e.g.

$$\text{marks} = 20.5\cdot(\text{study}) - 5\cdot(\text{sleep}) - 2.5\cdot(\text{parties}) - 1.5\cdot(\text{scrolling}) + b$$

This is exactly a **multiple linear regression**: $y = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b$. The middle node does the **summation**. The **bias** b plays the role of β_0 (and the error term handles deviations) in regression $y = \beta_0 + \beta_1x_1 + \beta_2x_2 + \varepsilon$. The bias has **no input** (think of its input as a constant 1) but it has a **weight**.

16.4 Multi-layer networks

In a **multi-layer network**, neurons are arranged in layers: the input and output layers are separated by one or more **hidden layers**. This layer-wise architecture is called a **feed-forward network** (information flows forward, input → output).



- Typically every node is connected to **every** node in the next layer (fully connected). If a connection is missing, there is no relationship between those nodes.
- More than one hidden layer → a **deep neural network**.
- **Architecture** = how many hidden layers and how many nodes in each (e.g. 1 input – 2 hidden nodes – 1 output; or input – hidden – hidden – output). Deeper networks often generalise better ("better generalisation with depth").

16.5 Anatomy of a neural network

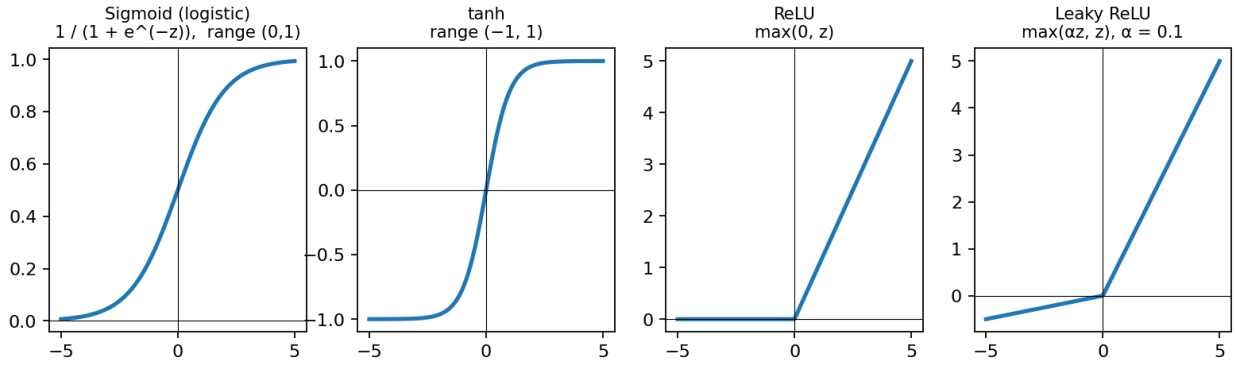
Component	Role
Inputs and weights	Each input is multiplied by a weight; plus a bias
Activation function	The function inside each node (applied to the weighted sum). Gives non-linearity
Loss function	Measures how wrong the predictions are; training minimises it (optimisation)
Output units	How many output nodes and which output function – depends on the task

Why not just a straight line? The sigmoid

For a regression output, a linear sum $w_1x_1 + w_2x_2 + \ldots$ is fine. But if the output is categorical (yes/no, 0/1), a straight line does not work. So we use a **logistic curve** that goes between 0 and 1 – in neural-network language this is the **sigmoid** function.

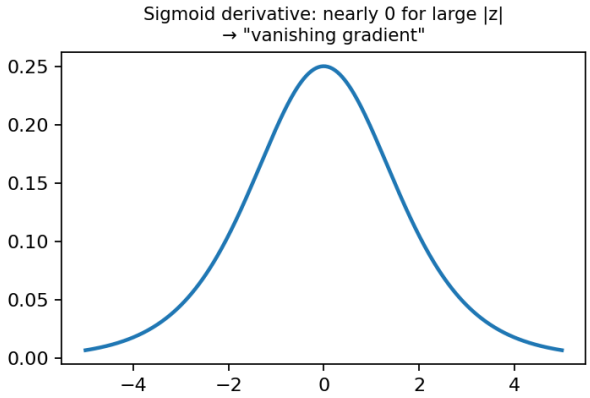
16.6 Activation functions

An activation function should give **non-linearity**, and its gradient should stay reasonably large during training. Common choices:



Function	Formula	Notes
Sigmoid (logistic)	$1 / (1 + e^{-z})$	Output in (0, 1). Derivative ≈ 0 for most of the domain → **vanishing gradient** during back-propagation
tanh (hyperbolic tangent)	tanh(z)	Output in (−1, 1); similar S-shape, centred at 0
ReLU (Rectified Linear Unit)	max(0, z)	A linear unit through the origin, but "rectified": negative values become 0. Most widely used and currently most successful; tends to work better than tanh
Leaky ReLU	max(αz, z), small α	Derivative is non-zero everywhere (fixes "dying ReLU"); results are not always consistent; α must be chosen

ReLU's two big advantages: (1) no vanishing gradient when z > 0 (slope is 1); (2) **sparsity** – when z < 0 the output is exactly 0, so many neurons are "off".



How the weights are estimated (**back-propagation**) will be covered next time – keep "vanishing gradient" in mind for that.

16.7 Loss function

Assume the observations are independent. Training **maximises the likelihood** (equivalently the log-likelihood, since log is one-to-one). So the loss function is the **negative log-likelihood**:

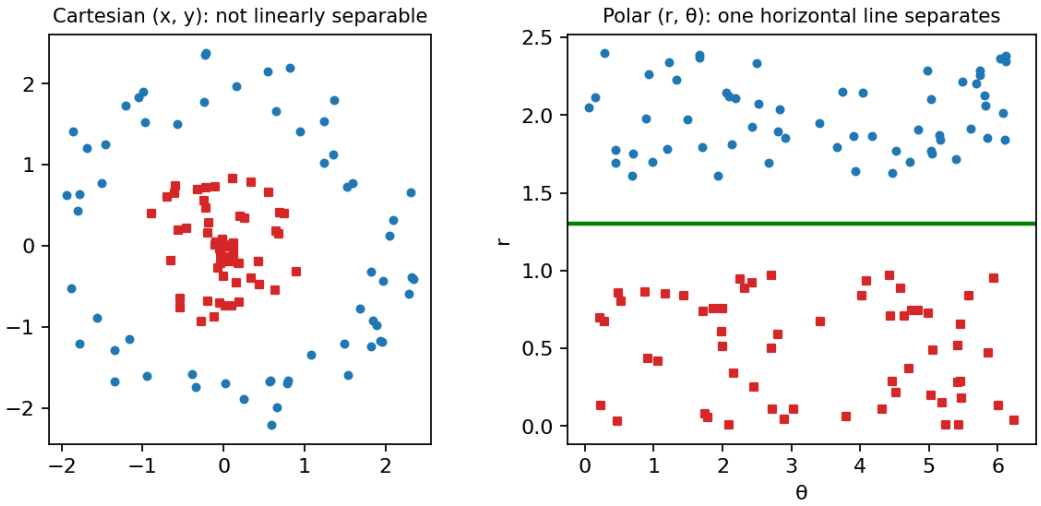
- If y is **Normal** (continuous output) → the loss becomes the **mean squared error (MSE)**.
- If y is **Bernoulli** (binary output) → the loss becomes the **(binary) cross-entropy**.

16.8 Output units

Output type	Output distribution	Output layer (activation)	Loss function	Example
Binary	Bernoulli	Sigmoid (1 node)	Binary cross-entropy	yes / no
Discrete (many classes)	Multinomial	Softmax (one node per class)	Cross-entropy	iris: 3 nodes – setosa, versicolor, virginica
Continuous	Gaussian	Linear	MSE	marks, mpg
Continuous	Arbitrary	(e.g. mixture / GANs)	Various	advanced

So the number of output nodes matters: a regression task (predict marks) has **1** output node; a 3-class classification (iris) has **3** output nodes.

16.9 Representation matters



The same data in Cartesian vs polar coordinates

With (x, y) coordinates the classes (inside circle vs outside ring) cannot be split by a line. In **polar** coordinates (r, θ) one horizontal line separates them. Finding a good representation is the key – and neural networks **learn** the representation themselves.

Approach	Pipeline
Rule-based systems	Input → hand-designed program → output
Classical machine learning	Input → **hand-designed features** → mapping from features → output
Representation learning	Input → **learned features** → mapping → output
Deep learning	Input → simple features → more abstract features (many layers) → mapping → output

Beyond linear models: linear models fit efficiently but have limited capacity. Traditional ML uses $w^T s(x)$, where s is a non-linear transform that must be **manually engineered** (domain-specific). Kernel methods (like RBF SVM) are one fixed choice. In **deep learning**, the feature transform is **learned automatically**: give an image, and the network finds the features (creates "embeddings") by itself – that is the beauty of neural networks.

16.10 Applications mentioned in class

- **Handwriting recognition:** write a letter by hand, take a photo, and it becomes a Word document. Classic dataset: **MNIST** (hand-written digits).
- **Image recognition:** is this a person, a car or an animal?
- **Smart traffic lights:** a camera detects an ambulance (siren and lights) and turns the light green.
- **Attendance by face recognition:** the lecturer's team at the university trained a model that recognises students entering the lecture hall and marks attendance – very accurate.
- **Layers learn a hierarchy:** first hidden layer → edges; second → corners and contours; third → object parts; then the whole object. Sometimes we cannot explain what a layer does – it is like a **black box**.
- **Transfer learning:** a model already trained on one task (e.g. digits) can be reused for another (e.g. identifying trees) – we transfer the knowledge.

Next time: exercises on neural networks, estimating weights by back-propagation. Check the LMS for the lab sheet (Monday), homework, videos and group-project instructions.

17. Key takeaways and R cheat-sheet

- An SVM chooses the **maximum-margin** decision boundary (the line in the middle).
- **Support vectors** are the observations that decide the boundary.
- The **soft margin** allows overlap through slack variables ξ; **C** controls the penalty (large C → narrow margin, overfit risk; small C → wide margin, underfit risk).
- The dual problem uses only inner products → the **kernel trick** gives non-linear boundaries without building the transformed variables.
- Multi-class SVM = **one-vs-one** sub-models + majority vote ($K(K-1)/2$ models).
- **ε-SVR** ignores errors inside an ε-wide tube; points outside are support vectors.
- Scale the data, tune with cross-validation on the training set, use suitable metrics, keep the test set untouched.
- A more flexible kernel is **not automatically** a better model (mtcars example!).
- Neural networks: input – hidden – output layers; perceptron = weighted sum + bias + activation; ReLU is the most common activation.

Task	R code (package e1071)
Classification SVM	svm(y ~ ., data = train, kernel = "linear" / "radial", cost = 1)
Regression SVR	svm(y ~ ., data = train, type = "eps-regression", kernel = "radial", epsilon = 0.1)
Default settings	type = C-classification (factor y), kernel = radial, cost = 1, gamma = 1/(number of predictors), epsilon = 0.1, scale = TRUE
Predict	predict(model, newdata = test)
Confusion matrix, accuracy	table(Observed = test$y, Predicted = pred); mean(pred == test$y)
Number of support vectors	model$tot.nSV (also shown by print(model))
Tune with 5-fold CV	tune(svm, y ~ ., data = train, ranges = list(cost = 2^(-1:5), gamma = 2^(-4:2)), tunecontrol = tune.control(cross = 5))
Best model / parameters	tuned$best.model ; tuned$best.parameters
Probabilities	svm(..., probability = TRUE); predict(m, test, probability = TRUE)
Class weights	svm(..., class.weights = c(A = 2, B = 1))
Plot boundary (2 predictors)	plot(model, data) or plot(model, data, x2 ~ x1)
RMSE / MAE	sqrt(mean((obs - pred)^2)) ; mean(abs(obs - pred))
Train/test split	ids <- sample(1:nrow(d), 0.8*nrow(d)); train <- d[ids,]; test <- d[-ids,]